



AIAA 3111 - Introduction to Data Mining

Pattern Mining: Advanced
Methods

Dr. Hao Liu



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Pattern Mining: Advanced Methods

- ☐ Mining Multi-level and multi-dimension Patterns 
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Summary

Mining Multiple-Level Frequent Patterns

- Items often form hierarchies

- Milk: Dairyland 2% milk
- Bread: Wonder wheat bread

Uniform support

Level 1
min_sup = 5%

Level 2
min_sup = 5%

Milk
[support = 10%]

2% Milk
[support = 6%]

Skim Milk
[support = 2%]

Reduced support

Level 1
min_sup = 5%

Level 2
min_sup = 1%

- Min-support thresholds for items following concept hierarchies
 - **Uniform** min-support across multiple levels (reasonable?)
 - **Level-reduced** min-support:
 - Items at the lower level are expected to have lower support
- Efficient mining: *Shared* multi-level mining
 - Use the lowest min-support to pass down the set of candidates

Redundancy Filtering at Mining Multi-Level Associations

- ❑ Redundancy filtering: redundant due to “ancestor” relationships
 - ❑ milk $\Rightarrow$ wheat bread [support = 8%, confidence = 70%] (1)
 - ❑ 2% milk $\Rightarrow$ wheat bread [support = 2%, confidence = 72%] (2)
 - ❑ Suppose the 2% milk sold is about $\frac{1}{4}$ of milk sold in gallons
 - ❑ (2) should be able to be “derived” from (1)
- ❑ **In general:** A rule is *redundant* if its support is close to the “expected” value, according to its “ancestor” rule, and it has a similar confidence as its “ancestor”

Q: If rule (1) is an ancestor of rule (2), which one to prune?

Customized Min-Supports for Different Kinds of Items

- ❑ We have used the same min-support threshold for all the items or item sets to be mined in each association mining
- ❑ In reality, some items (e.g., diamond, watch, ...) are valuable but less frequent
- ❑ It is necessary to have customized min-support settings for different kinds of items
- ❑ One Method: Use **group-based “individualized” min-support**
 - ❑ E.g., {diamond, watch}: 0.05%; {bread, milk}: 5%; ...
 - ❑ How to mine such rules efficiently?
 - ❑ Existing scalable mining algorithms can be easily extended to cover such cases

Mining Multi-Dimensional Associations

- ❑ **Single-dimensional** rules (e.g., items are all in “product” dimension)
 - ❑ $\text{buys}(X, \text{“milk”}) \Rightarrow \text{buys}(X, \text{“bread”})$
- ❑ **Multi-dimensional** rules (i.e., items in ≥ 2 dimensions or predicates)
 - ❑ Inter-dimension association rules (*no repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{occupation}(X, \text{“student”}) \Rightarrow \text{buys}(X, \text{“coke”})$
 - ❑ Hybrid-dimension association rules (*repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{buys}(X, \text{“popcorn”}) \Rightarrow \text{buys}(X, \text{“coke”})$
- ❑ Attributes can be categorical or numerical
 - ❑ Categorical Attributes (e.g., *profession*, *product*: no ordering among values): Data cube for inter-dimension association
 - ❑ Quantitative Attributes: Numeric, implicit ordering among values—discretization, clustering, and gradient approaches

Rare Patterns vs. Negative Patterns

❑ Rare patterns

- ❑ Very low support but interesting (e.g., buying Rolex watches)
- ❑ How to mine them? Setting individualized, group-based min-support thresholds for different groups of items

❑ Negative patterns

- ❑ Negatively correlated: Unlikely to happen together
- ❑ Ex.: Since it is unlikely that the same customer buys both a **Ford Expedition** (an SUV car) and a **Ford Fusion** (a hybrid car), buying a **Ford Expedition** and buying a **Ford Fusion** are likely negatively correlated patterns
- ❑ How to define negative patterns?

Defining Negative Correlated Patterns

- A support-based definition
 - If itemsets A and B are both frequent but rarely occur together, i.e.,
 - $\text{sup}(A \cup B) \ll \text{sup}(A) \times \text{sup}(B)$
 - Then A and B are negatively correlated
- Is this a good definition for large transaction datasets?
- Ex.: Suppose a store sold two needle packages A and B 100 times each, but only one transaction contained both A and B
 - When there are in total 200 transactions, we have
 - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) \ll s(A) \times s(B)$
 - But when there are 10^5 transactions, we have
 - $s(A \cup B) = 1/10^5, s(A) \times s(B) = 1/10^3 \times 1/10^3, s(A \cup B) > s(A) \times s(B)$
- What is the problem here?
 - Null transactions: The support-based definition is not null-invariant!

Does this remind you the definition of *lift*?

Defining Negative Correlation: Need Null-Invariance in Definition

- A good definition on negative correlation should be null-invariant!
 - Intuition: Whether two itemsets A and B are negatively correlated should not be influenced by the number of null-transactions
- A Kulczynski measure-based definition
 - If itemsets A and B are frequent but $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$, then A and B are negatively correlated
- For the same needle package problem:
 - No matter there are in total 200 or 10^5 transactions
 - If $\epsilon = 0.01$, we have
$$(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 = (0.01 + 0.01)/2 < \epsilon$$

ϵ is a negative pattern threshold

Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining 
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns
- ❑ Summary

Why Constraint-Based Mining?

- ❑ Pattern mining in practice: Often a user-guided, **interactive** process
 - ❑ User directs what to be mined using a **data mining query language** (or a graphical user interface), **specifying various kinds of constraints**

- ❑ What is constraint-based mining?
 - ❑ Mine together with user-provided constraints

- ❑ Why constraint-based mining?
 - ❑ User flexibility: User provides **constraints** on what to be mined
 - ❑ Optimization: System explores such constraints for mining efficiency
 - ❑ E.g., Push constraints deeply into the mining process

Various Kinds of User-Specified Constraints in Data Mining

- ❑ **Knowledge type constraint**—Specifying what kinds of knowledge to mine
 - ❑ E.g.: Classification, association, clustering, outlier finding, ...
- ❑ **Data constraint**—using SQL-like queries
 - ❑ E.g.: Find products sold together in NY stores this year
- ❑ **Dimension/level constraint**—similar to projection in relational database
 - ❑ E.g.: In relevance to region, price, brand, customer category
- ❑ **Interestingness constraint**—various kinds of thresholds
 - ❑ E.g.: Strong rules: $\text{min_sup} \geq 0.02$, $\text{min_conf} \geq 0.6$, $\text{min_correlation} \geq 0.7$
- ❑ **Rule (or pattern) constraint**  **The focus of this study**
 - ❑ E.g.: Small sales (price < \$10) triggers big sales (sum > \$200)

Pattern Space Pruning with Pattern Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **anti-monotone**
 - If an itemset S **violates** constraint c , so does any of its superset
 - That is, mining on itemset S can be terminated
- Ex. 1: $c_1: \text{sum}(S.\text{price}) \leq v$ is **anti-monotone**
- Ex. 2: $c_2: \text{range}(S.\text{profit}) \leq 15$ is **anti-monotone**
 - Itemset ab violates c_2 ($\text{range}(ab) = 40$)
 - So does every superset of ab
- Ex. 3. $c_3: \text{sum}(S.\text{Price}) \geq v$ is **not anti-monotone**
- Ex. 4. Is $c_4: \text{support}(S) \geq \sigma$ anti-monotone?
 - Yes! Apriori pruning is essentially pruning with an anti-monotonic constraint!

Note: item.price > 0
Profit can be negative

Pattern Monotonicity and Its Roles

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is *monotone*: If an itemset S satisfies the constraint c , so does any of its superset
 - That is, we do not need to check c in subsequent mining
 - Not as beneficial as anti-monotone
- E.g. 1: $c_1: \text{sum}(S.\text{Price}) \geq v$ is **monotone**
- E.g. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **monotone**
- E.g. 3: $c_3: \text{range}(S.\text{profit}) \geq 15$ is **monotone**
 - Itemset ab satisfies c_3
 - So does every superset of ab

Apriori for Pattern Anti-Monotone Constraint

Item	Price
1	1
2	2
3	3
4	4
5	5

Database D

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Scan D

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

C_1

F_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

Can be
chopped
early

F_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

C_3

itemset	sup
{2 3 5}	2

Scan D

F_3

itemset	sup
{2 3 5}	2

Min_sup=2
Constraint:
Sum{S.price} < 5

Convertible Constraints: Ordering Data in Transactions

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

- Convert tough constraints into (anti-)monotone by proper ordering of items in transactions
- Examine c_1 : $\text{avg}(S.\text{profit}) > 20$
 - Order items in (profit) value-descending order
 - $\langle a, g, h, b, f, d, c, e \rangle$
 - An itemset ab violates c_1 ($\text{avg}(ab) = 20$)
 - So does ab^* (i.e., ab -projected DB)
 - C_1 : **anti-monotone if patterns grow in the right order!**

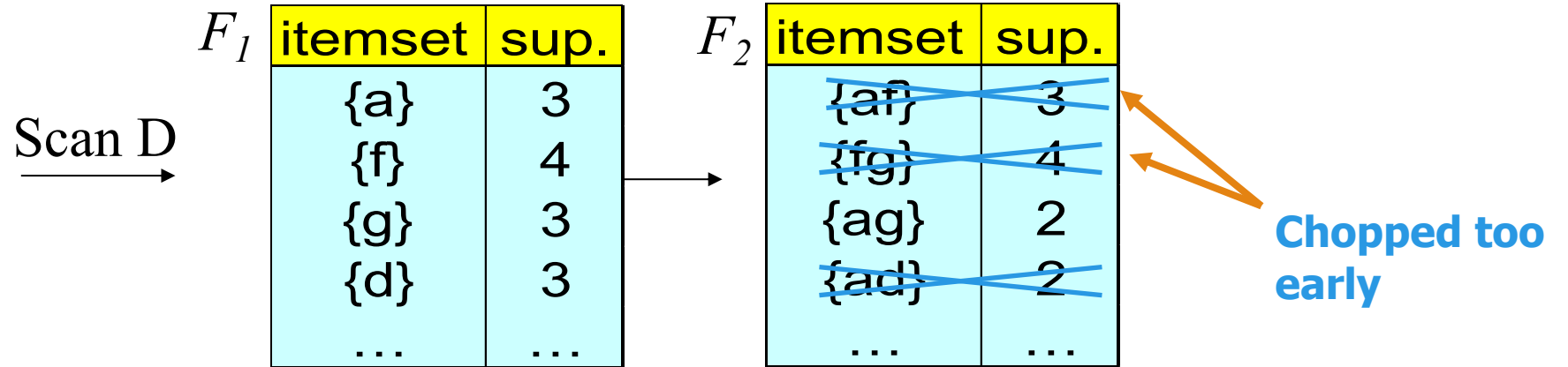
Can item-reordering work for Apriori?

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

constraint: $\text{avg}(\text{S.profit}) > 20$



- $\text{avg}(gf) = 12.5 < 20$, $\text{avg}(af) = 17.5 < 20$, $\text{avg}(ag) = 35 > 20$
- But $\text{avg}(agf) = 21.7 > 20$
- Apriori will not generate “agf” as a candidate

Data Space Pruning with Data Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **data anti-monotone**: In the mining process, if a data entry t cannot satisfy a pattern p under c , t cannot satisfy p 's superset either
 - Data space pruning: Data entry t can be pruned
- Ex. 1: $c_1: \text{sum}(S.\text{Profit}) \geq v$ is **data anti-monotone**
 - Let constraint c_1 be: $\text{sum}(S.\text{Profit}) \geq 25$
 - $T_{30}: \{b, c, d, f, g\}$ can be removed since none of their combinations can make an S whose sum of the profit is ≥ 25
- Ex. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **data anti-monotone**
 - Consider $v = 5$ but every item in a transaction, say T_{50} , has a price higher than 10
- Ex. 3: $c_3: \text{range}(S.\text{Profit}) > 25$ is **data anti-monotone**

Data Space Pruning Should Be Explored Recursively

TID	Transaction	Item	Profit
10	a, b, c, d, f, h	a	40
20	b, c, d, f, g, h	b	0
30	b, c, d, f, g	c	-20
40	a, c, e, f, g	d	-15
min_sup = 2		e	-30
		f	-10
		g	20
		h	5

- ❑ Example. $c_3: \text{range}(S.\text{Profit}) > 25$
 - ❑ We check b's projected database
 - ❑ But item "a" is infrequent (sup = 1)
 - ❑ After removing "a (40)" from T_{10}
 - ❑ T_{10} cannot satisfy c_3 any more
 - ❑ Since "b (0)" and "c (-20), d (-15), f (-10), h (5)"
 - ❑ By removing T_{10} , we can also prune "h" in T_{20}

b's-proj. DB

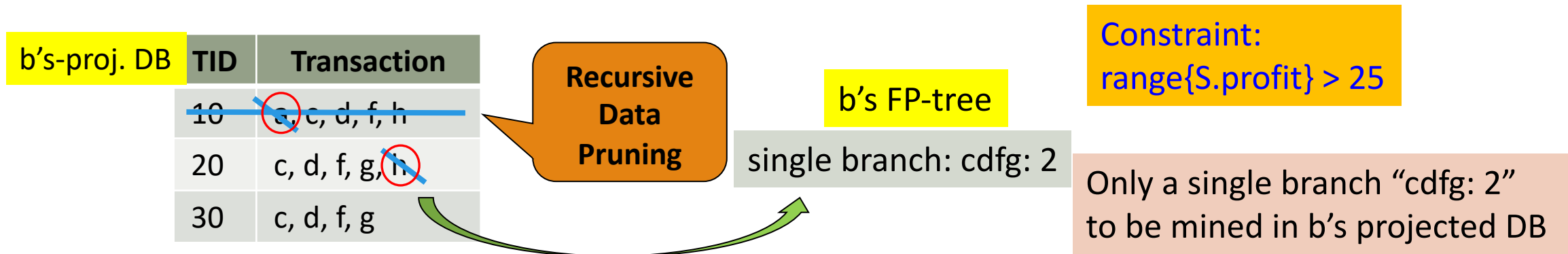


TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

b's-proj. DB

TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

Data Space Pruning Explored Recursively (Continued)



- Note: c_3 prunes T_{10} effectively only after "a" is pruned (by min-sup) in b's projected DB

Succinctness: Pruning Both Data and Pattern Spaces

- Succinctness: If the constraint c can be enforced by directly manipulating the data
- E.g. 1: To find those patterns containing item i
 - Mine only i -projected DB (data space pruning)
- E.g. 2: To find those patterns without item i
 - Remove i from DB and then mine (pattern space pruning)
- E.g. 3: $c_3: \min(S.Price) \leq v$ is succinct
 - Start with only items whose price $\leq v$ and remove transactions with high-price items only (pattern + data space pruning)
- E.g. 4: $c_4: \sum(S.Price) \geq v$ is not succinct
 - It cannot be determined beforehand since sum of the price of itemset S keeps increasing

Constrained FP-Growth: Push a Succinct Constraint Deep

Item	Price
1	1
2	2
3	3
4	4
5	5

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Remove
infrequent
length 1

TID	Items
10	1 3
20	2 3 5
30	1 2 3 5
40	2 5

Min_sup=2

Constraint:

$\min\{S.\text{price}\} \leq 2$

No Need to project on 3 or 5

1-Projected DB

TID	Items
10	3 4
30	2 3 5

2-Projected DB

TID	Items
20	3 5
30	1 3 5
40	5

Different Kinds of Constraints Lead to Different Pruning Strategies

- In summary, constraints can be categorized as **pattern space pruning** constraints vs. **data space pruning** constraints

Pattern space pruning constraints	Data space pruning constraints
■ Anti-monotonic: If constraint c is violated, its further mining can be terminated ■ Monotonic: If c is satisfied, no need to check c again ■ Convertible: c can be converted to monotonic or anti-monotonic if items can be properly ordered in processing ■ Succinct: If the constraint c can be enforced by directly manipulating the data	■ Data succinct: Data space can be pruned at the initial pattern mining process ■ Data anti-monotonic: If a transaction t does not satisfy c, then t can be pruned to reduce data processing effort

How to Handle Multiple Constraints?

- It is beneficial to use multiple constraints in pattern mining
- But different constraints may require potentially conflicting item-ordering
 - If there exists conflict ordering between c_1 and c_2
 - Try to sort data and enforce *one constraint* first (which one?)
 - Then enforce the other constraint when mining the projected databases
- E.g. c_1 : $\text{avg}(S.\text{profit}) > 20$, and c_2 : $\text{avg}(S.\text{price}) < 50$
 - Assume c_1 has more pruning power
 - Sort in profit descending order and use c_1 first
 - For each project DB, sort trans. in price ascending order and use c_2 at mining

Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns 
- ❑ Mining Subgraph Patterns
- ❑ Summary

Sequential Pattern Mining

- ❑ What kind of patterns are sequential?
- ❑ Sequential: The order really matters
 - ❑ You cannot swap two items in a sequence and have the same sequence
- ❑ Example: The English language is sequential: Subject → Verb → Object

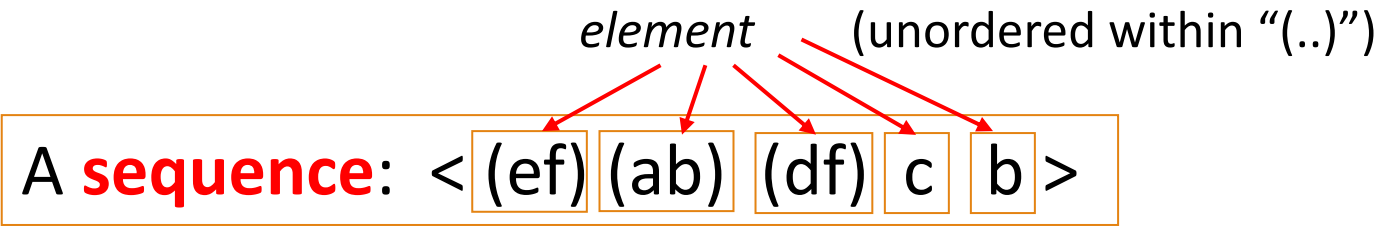
- ❑ Other points:
 - ❑ For Sequential Pattern Mining, the time which the items occur is **not** considered
 - ❑ Time Series Analysis does take into account the time in which an item occurred

Sequential Pattern Examples

- Application of Sequential pattern Mining
 - **Customer shopping:** Purchase a laptop first, then a digital camera, and then a smartphone
 - **Medical treatments:** Go to see a doctor, get drugs, doctor monitors progress, doctor reacts accordingly → more/less drugs
 - **Natural disasters:** Before the disaster, during the disaster, after the disaster
 - **Scientific Experiments:** Step 1, Step 2, Step 3
 - **Stocks Markets:** A set of stocks go up and down together
 - **Biological sequences, DNA /Protein:** If you change the order of a protein, it likely results in a different gene

Sequential Pattern and Sequential Pattern Mining

- Sequential pattern mining: Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- An element may contain a set of items (also called events)

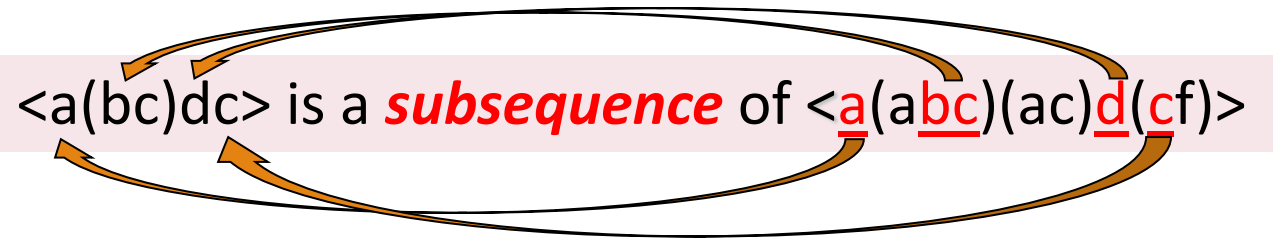
* Items within an element are **unordered** and we list them alphabetically

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

Sequential Pattern and Sequential Pattern Mining

- **Sequential pattern mining:** Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- Given support threshold $min_sup = 2$, <(ab)c> is a **sequential pattern**

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

Sequential Pattern Mining Algorithms

- ❑ Algorithm requirement: Efficient, scalable, finding complete set, incorporating various kinds of user-specific constraints
- ❑ The Apriori property still holds: If a subsequence s_1 is infrequent, none of s_1 's super-sequences can be frequent
- ❑ Representative algorithms
 - ❑ **GSP** (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: Apriori-Based Sequential Pattern Mining

- Initial candidates: All 8-singleton sequences
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
- Scan DB once, count support for each candidate

SID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

$min_sup = 2$

Cand.	sup
$\langle a \rangle$	3
$\langle b \rangle$	5
$\langle c \rangle$	4
$\langle d \rangle$	3
$\langle e \rangle$	3
$\langle f \rangle$	2
$\langle g \rangle$	1
$\langle h \rangle$	1



GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: Apriori-Based Sequential Pattern Mining

- Example: Generate length-2 candidate sequences

singleton * singleton – Total: (6 * 6)

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

min_sup = 2

Cand.	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



Sets (unordered) – Total: (6*5) / 2

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Apriori Pruning

- w/o pruning
(includes g and h):

$$8*8 + 8*7/2 = 92$$

length-2 candidates

- w/ pruning:

$$6*6 + 6*5/2 = 51$$

length-2 candidates

GSP Mining and Pruning

			<u>length</u>
5 th scan: 1 cand. 1 length-5 seq. pat.	<(bd)cba>	☐ Remove Candidates not in DB or Candidates < min_sup	5
4 th scan: 8 cand. 7 length-4 seq. pat.	<abba> <(bd)bc> ...		4
3 rd scan: 46 cand. 20 length-3 seq. pat. 20 cand. not in DB at all	<abb> <aab> <aba> <baa> <bab> ...		3
2 nd scan: 51 cand. 19 length-2 seq. pat. 10 cand. not in DB at all	<aa> <ab> ... <af> <ba> <bb> ... <ff> <(ab)> ... <(ef)>		2
1 st scan: 8 cand. 6 length-1 seq. pat.	<a> <c> <d> <e> <f> <g> <h>		1

$$6*6 + 6*5/2 = 51$$

The GPS algorithm

- ☐ Repeat, starting at k = 1 until k <= length
 - ☐ Scan DB to find “length-k” frequent sequences
 - ☐ Generate “length-(k+1)” candidate sequences from “length-k” frequent sequences using **Apriori**
 - ☐ set k = k+1
- ☐ Until no frequent sequence or no candidate can be found

min_sup = 2

SID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

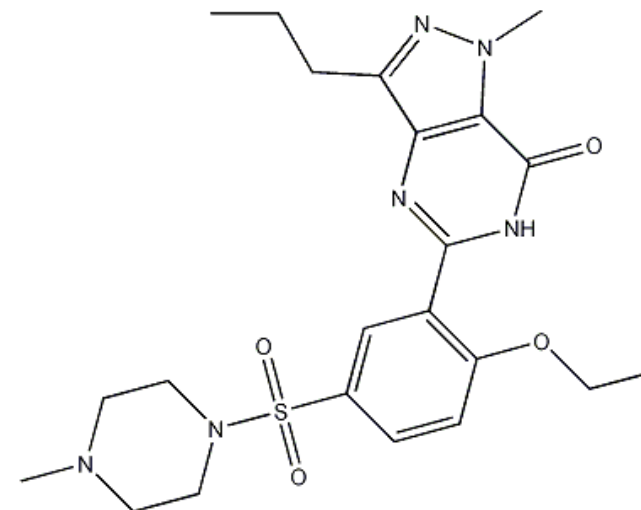
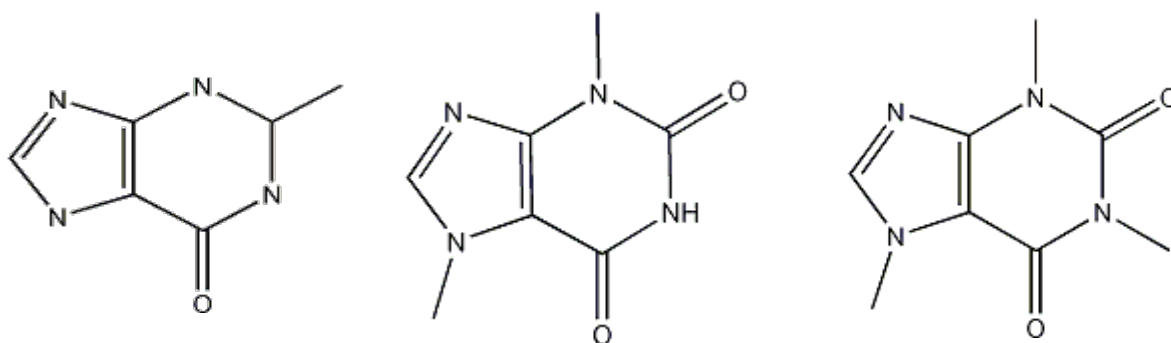
Pattern Mining: Advanced Methods

- ❑ Mining Multi-level and multi-dimension Patterns
- ❑ Constraint-based Pattern Mining
- ❑ Mining Sequential Patterns
- ❑ Mining Subgraph Patterns 
- ❑ Summary

What Is Graph Pattern Mining?

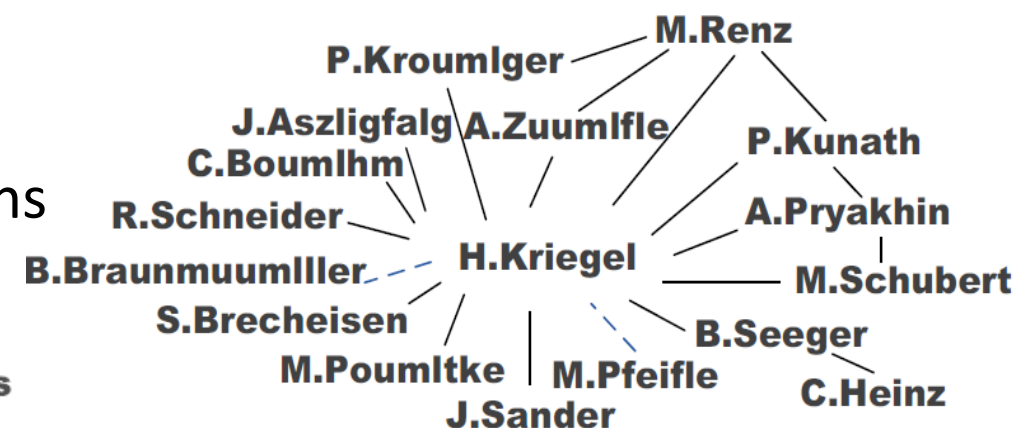
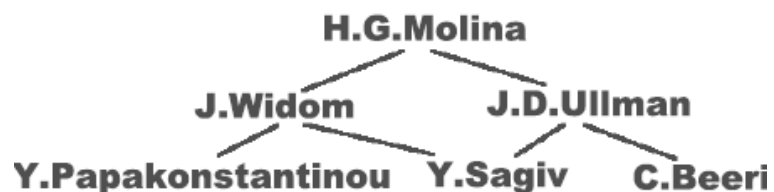
- Chem-informatics:

- Mining frequent chemical compound structures



- Social networks, web communities, tweets, ...

- Finding frequent research collaboration subgraphs



Frequent (Sub)Graph Patterns

- Given a labeled graph dataset $D = \{G_1, G_2, \dots, G_n\}$, the supporting graph set of a subgraph g is $D_g = \{G_i \mid g \subseteq G_i, G_i \in D\}$

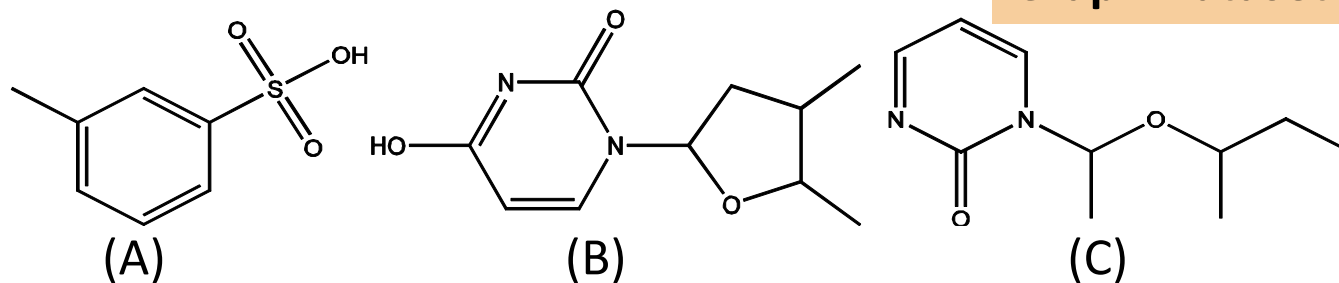
- $\text{support}(g) = |D_g| / |D|$

- A (sub)graph g is **frequent** if $\text{support}(g) \geq \text{min_sup}$

- Ex.: Chemical structures

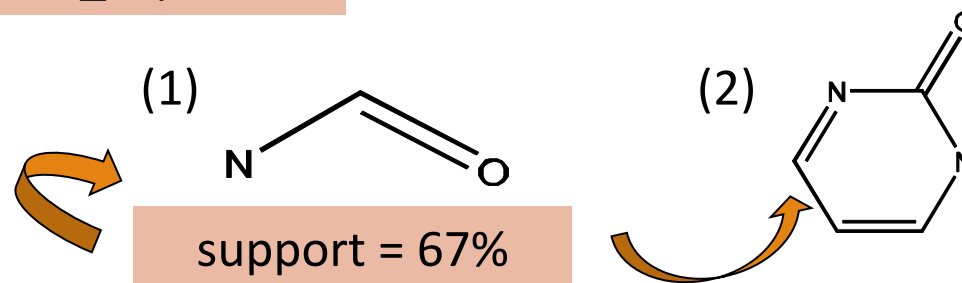
- Alternative:

- Mining frequent subgraph patterns from a single large graph or network



min_sup = 2

Frequent Graph Patterns

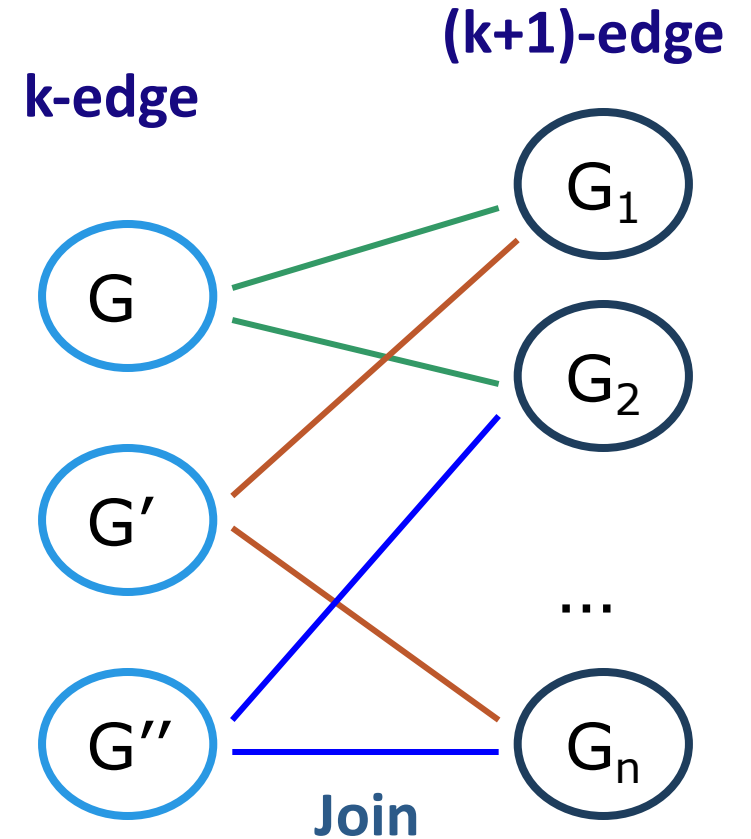


Applications of Graph Pattern Mining

- ❑ Bioinformatics
 - ❑ Gene networks, protein interactions, metabolic pathways
- ❑ Chem-informatics: Mining chemical compound structures
- ❑ Social networks, web communities, tweets, ...
- ❑ Cell phone networks, computer networks, ...
- ❑ Web graphs, XML structures, Semantic Web, information networks
- ❑ Software engineering: Program execution flow analysis
- ❑ Building blocks for graph classification, clustering, compression, comparison, and correlation analysis
- ❑ Graph indexing and graph similarity search

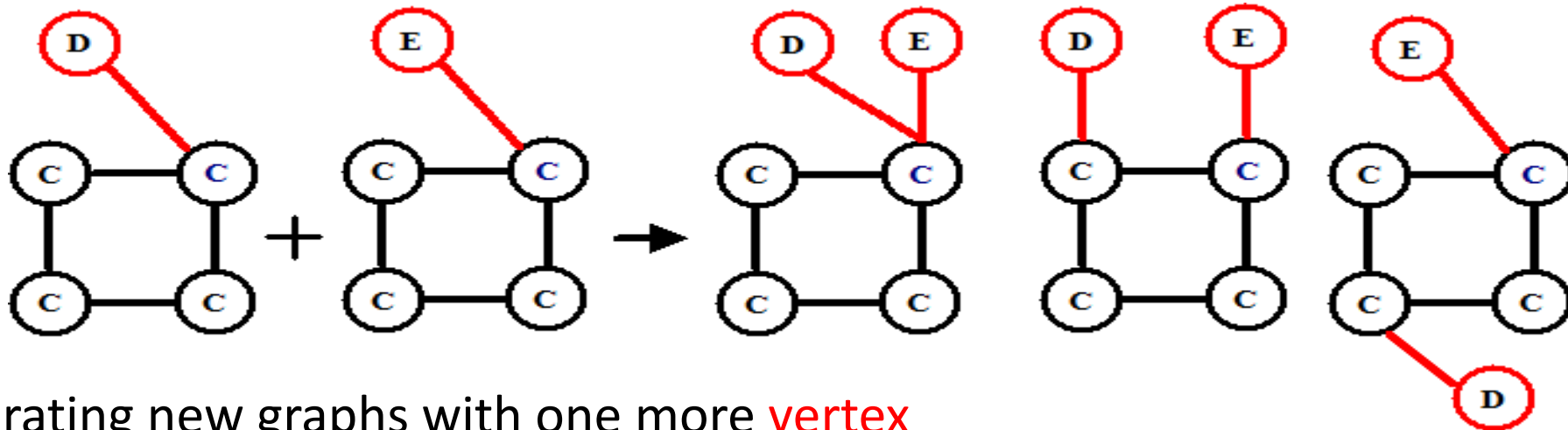
Apriori-Based Approach

- The Apriori property (anti-monotonicity): A size- k subgraph is **frequent** if and only if all of its **subgraphs are frequent**
- A candidate size- $(k+1)$ edge/vertex subgraph is generated if its corresponding two k -edge/vertex subgraphs are frequent
- Iterative mining process:
 - Candidate-generation $\rightarrow$ candidate pruning $\rightarrow$ support counting $\rightarrow$ candidate elimination



Candidate Generation: Vertex Growing vs. Edge Growing

- ❑ Methodology: Breadth-search, Apriori joining two size- k graphs
 - ❑ Many possibilities at generating size- $(k+1)$ candidate graphs



- ❑ Generating new graphs with one more **vertex**
 - ❑ AGM (Inokuchi, Washio, & Motoda, PKDD'00)
- ❑ Generating new graphs with one more **edge**
 - ❑ FSG (Kuramochi & Karypis, ICDM'01)
- ❑ Performance shows *via edge growing* is more efficient

Pattern-Growth Approach

- Depth-first growth of subgraphs from k -edge to $(k+1)$ -edge, then $(k+2)$ -edge subgraphs

- Major challenge

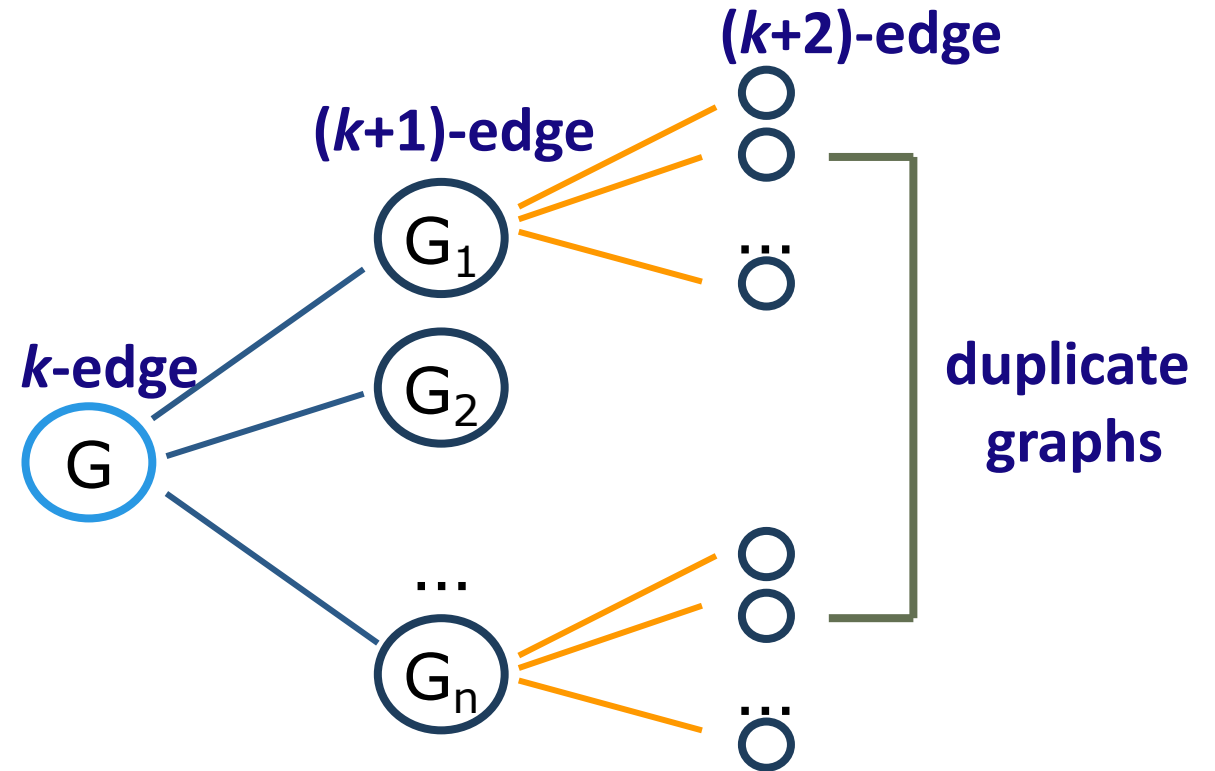
- Generating many duplicate subgraphs

- Major idea to solve the problem

- Define an order to generate subgraphs

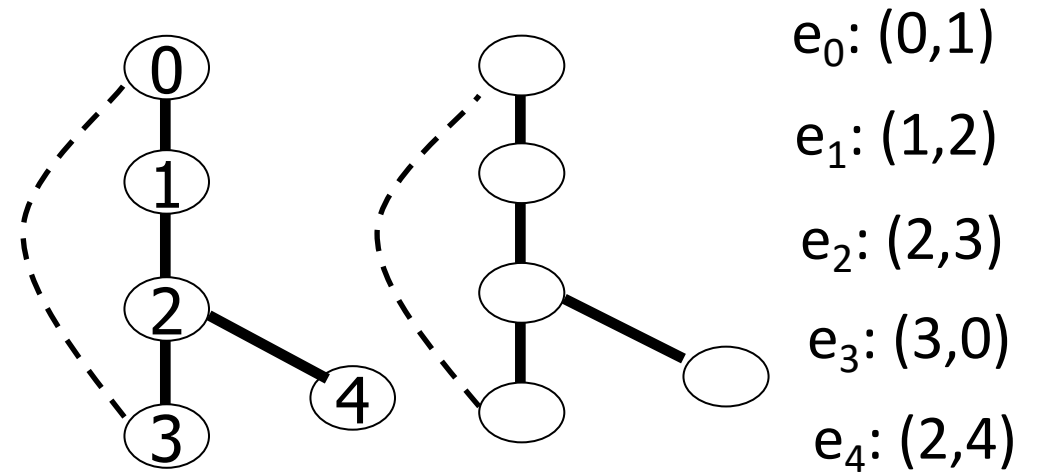
- DFS spanning tree: Flatten a graph into a sequence using depth-first search

- gSpan (Yan & Han, ICDM'02)



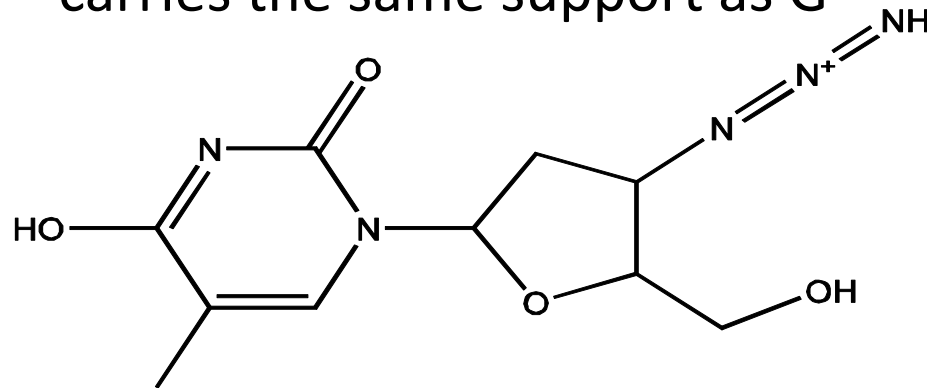
gSPAN: Graph Pattern Growth in Order

- ❑ **Right-most path extension** in subgraph pattern growth
 - ❑ Right-most path: The path from root to the right-most leaf (choose the vertex with the **smallest** index at each step)
 - ❑ Reduce generation of duplicate subgraphs
- ❑ **Completeness:** The enumeration of graphs using right-most path extension is complete
- ❑ DFS code: Flatten a graph into a sequence using depth-first search



Mining Closed Graph Patterns

- 2^n subgraphs $\rightarrow$ *closed frequent subgraphs*
- A frequent graph G is *closed* if there exists no supergraph of G that carries the same support as G

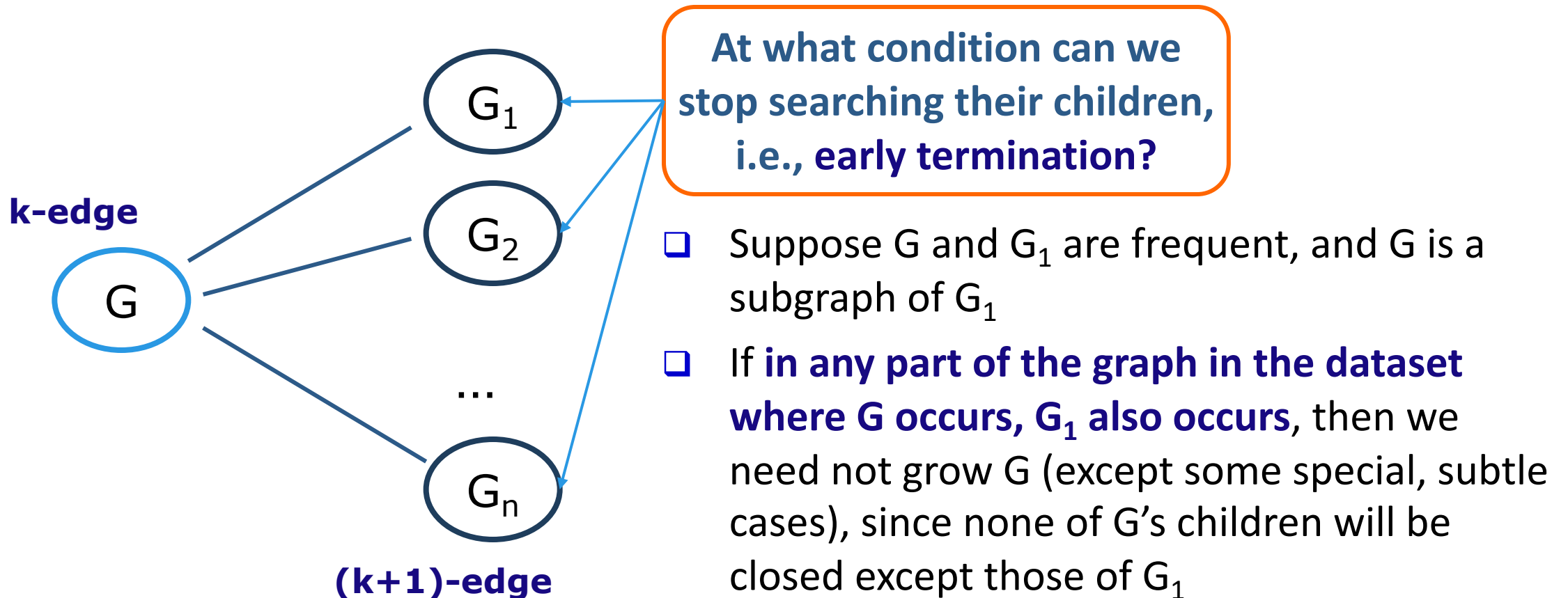


If this subgraph is *closed* in the graph dataset, it implies that none of its frequent super-graphs carries the same support

- *Lossless compression*: Does not contain non-closed graphs, but still ensures that the mining result is complete
- Algorithm CloseGraph: Mines closed graph patterns directly

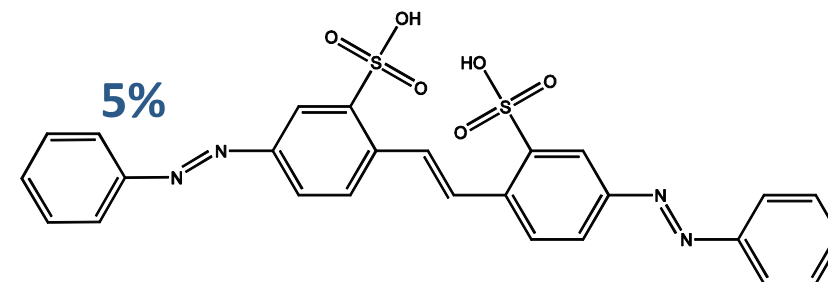
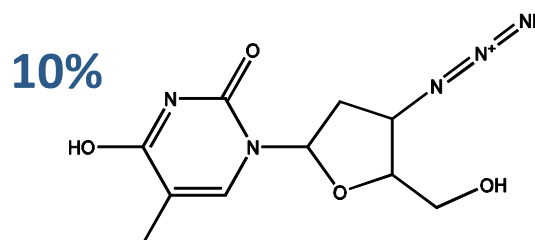
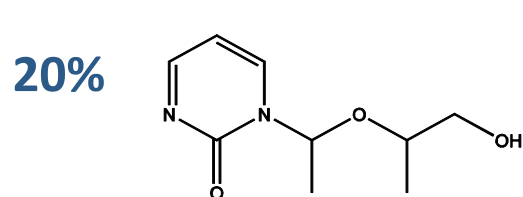
CloseGraph: Directly Mining Closed Graph Patterns

- CloseGraph: Mining closed graph patterns by extending gSpan (Yan & Han, KDD'03)

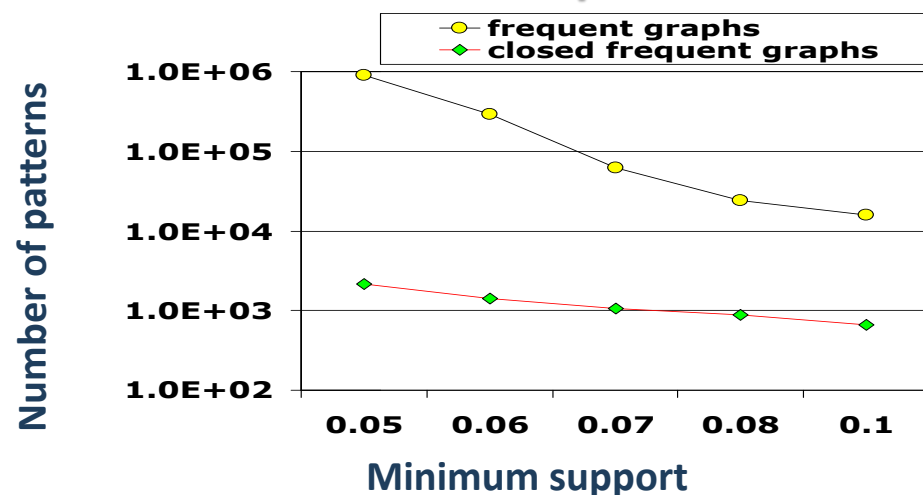


Experiment and Performance Comparison

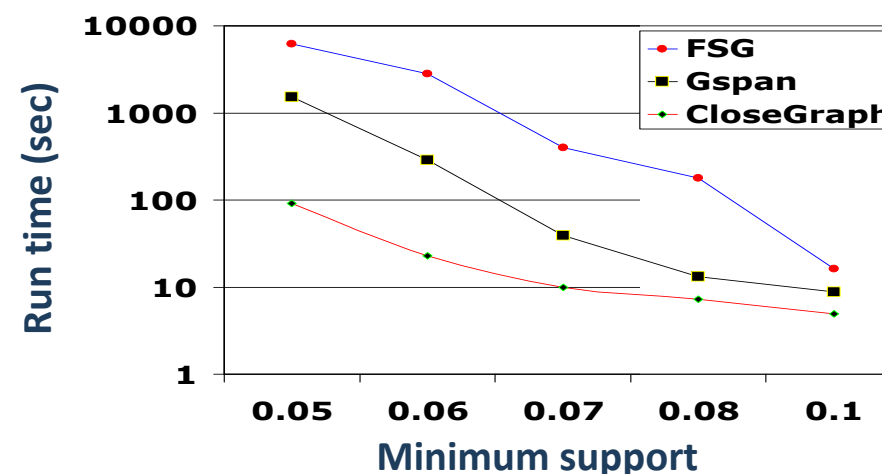
- ❑ The AIDS antiviral screen compound dataset from NCI/NIH
- ❑ The dataset contains 43,905 chemical compounds
- ❑ Discovered patterns: The smaller minimum support, the bigger and more interesting subgraph patterns discovered



of Patterns: Frequent vs. Closed



Runtime: Frequent vs. Closed

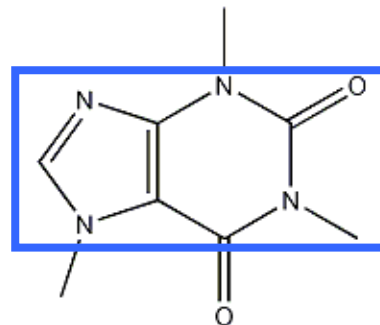
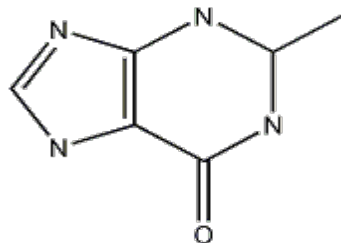


Application: Support Substructure Similarity Search

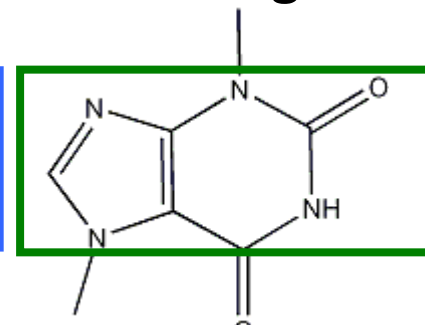
- Find graphs in a graph DB containing substructures similar to a given query graph

- Ex. Data: A chemical compound DB

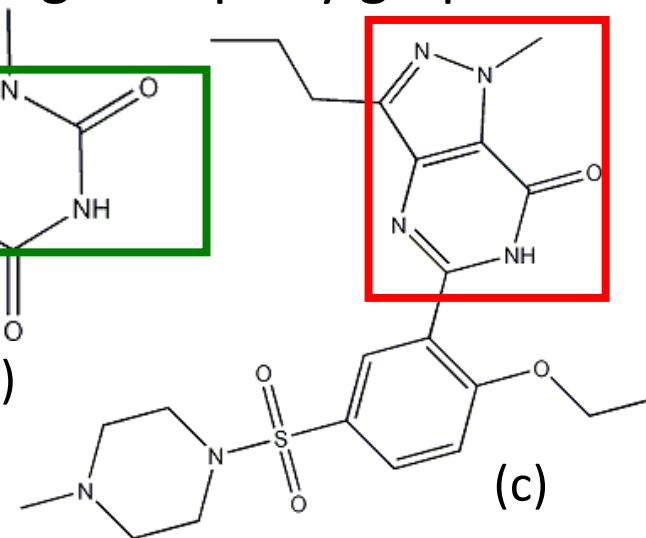
- A query graph q :



(a)



(b)



(c)

- How to do similarity search efficiently?

- Sequential scan + computing subgraph similarity – too costly!

- Build graph indices to support approximate search?

- Need an explosive number of subgraphs to cover all the *similar* subgraphs!

- An elegant solution (Yan, Yu, & Han, SIGMOD'05):

- Keep the graph index structure, but **select features** in the **query space**

Summary

- ☐ Mining Diverse Patterns
- ☐ Constraint-Based Frequent Pattern Mining
- ☐ Sequential Pattern Mining
- ☐ Graph Pattern Mining

Q & A



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)